Bilgisayar Programlama
2

Sözlük Veri Türü

- Dikkat: Sozluk tanımlanırken {} kullanılır

```
calisanlar = {"Ahmet":"Muhasebe","Ayşe":"Pazarlama","Fatma":"Satış"}
print(calisanlar["Ahmet"])
calisanlar["Mustafa"] = {"Güvenlik"}  #Ekleme
print(calisanlar)
calisanlar["Fatma"]="İnsan Kaynakları" #Değişiklik
print(calisanlar)

del calisanlar["Ahmet"]  #Silme
print(calisanlar)

print (calisanlar.keys())   #sözlükteki anahtarlar
print (calisanlar.values()) #sözlükteki değerler
print (calisanlar.items())  #sözlükteki tüm varlıklar
```

```
>>> telefon_defteri = {
"Ahmet": "0533 123 45 67",
"Salih": "0532321 54 76",
"Selin": "0533 333 33 33"
}
```

Sözlük Veri Türü

- bütün öğeleri silmek istersek
  - telefon_defteri.clear()
  - Calisanlar.clear()

Sözlük Veri Türü

- Get() işlevi ile bir değerin sözlükte olup olmadığı kontrol edilebilir
  - Yok ise bir mesaj oluşturulabilir..

```
personel = {"Ahmet":"Muhasebe","Ayşe":"Pazarlama","Fatma":"Satış"}

kisi = input("Sorgulanacak personelin ismini girin:")

print (personel.get(kisi,"personel bulunamadi.."))
```

Sözlük Veri Türü

- Hava durumu programı yazmak istiyoruz. Tasarımıza göre kullanıcı bir şehir adı girecek. Program da girilen şehre özgü hava durumu bilgilerini ekrana yazdıracak.

```
soru = raw_input("Şehrinizin adının tamamı küçük harflerle yazınız: ")

cevap = {
"istanbul":"gök gürültülü ve sağanak yağışlı",
"ankara":"açık ve güneşli",
"izmir":"bulutlu"
}

print cevap.get(soru,"Bu şehir için havadurumu bilgisi bulunmamaktadır.")
```

Sıralı Sözlükler

- Sözlüklerde herhangi bir sıra kavramı bulunmaz.

```
>>> a = {"ali": 1, "veli": 2, "selim": 3}
>>> print a

{'selim': 3, 'veli': 2, 'ali': 1}
```

- bazı durumlarda, bir sözlük yapısı içinde tanımladığınız öğelerin sırasını korumanız gerekebilir. Örneğin şöyle bir personel kaydı olsun;
- **>>>** personel = { 'Ahmet' : '19.01.2013',
  'Mehmet': '21.03.2013',
  'Selin' : '30.06.2013'
  }

Sıralı Sözlükler

- Herhangi bir şekilde sözlük yapısı içinde tanımladığınız öğelere, bu öğeleri tanımladığınız sıra ile erişmeniz gerekirse Python'daki 'OrderedDict'.
- OrderedDict, 'collections' adlı bir modül içinde yer alır.

```
>>> from collections import OrderedDict
```

- Sıralı sözlükler, normal sözlüklerle aynı metotlara sahipt

```
>>> personel.keys()

['Ahmet', 'Mehmet', 'Selin']

>>> personel.values()

['19.01.2013', '21.03.2013', '30.06.2013']

>>> print personel.get("Sedat", "Böyle biri yok!")

Böyle biri yok!
```

Fonksiyonlar

- Python'da fonksiyonları :
  - **def** fonksiyon_adi():
- önce def parçacığını, ardından da fonksiyonumuzun adını yazıyoruz.
  - Fonksiyon adlarında Türkçe karakter kullanmamanız gerekiyor.
  - Ayrıca fonksiyonları tanımlarken en sona parantez ve iki nokta üst üste işaretleri konmalıdır.

```
def selamlama():
  dil="Phyton programlama dili"
  print (dil)
```

Fonksiyonlar

- Fonksiyonun çalışması için;

```
def selamlama():
  dil="Phyton programlama dili"
  print (dil)
selamlama()
```

- fonksiyonun çalışmasını sağlayan şey, en son satırdaki selamlama() kodudur

Fonksiyonlar

- **bir fonksiyon def parçacığıyla başlar, gövdesindeki girintili alanın sonuna kadar devam eder.**

```
1   def tek():
2       print ("Girdiğiniz sayı bir tek sayıdır!")
3   def cift():
4       print ("Girdiğiniz sayı bir çift sayıdır!")
5   sayi = input("Lütfen bir sayı giriniz: ")
6
7   if int(sayi) % 2 == 0:
8       cift()
9   else:
10          tek()
```

Fonksiyonlarda Parametre Kullanımı

- Bir fonksiyon tanımlarken, fonksiyon adını belirledikten sonra bir parantez işareti kullanılır.
- Bu parantez işaretleri, örneklerde görüldüğü gibi hiçbir bilgi içermeyebilir.
- Parantezlerin içine yazılan bilgilere parametre adı verilir.

```
def selamla(isim):
    print ("merhaba, benim adım %s!" %isim)
selamla("mehmet")
```

- Mehmet argümanı fonksiyona parametre olarak gönderildi

Fonksiyonlarda Parametre Kullanımı

```
def selamla(isim):
  print ("merhaba, benim adım %s!" %isim)
selamla("mehmet","ahmet")
```

```
  File "main.py", line 3, in <module>
    selamla("mehmet","ahmet")
TypeError: selamla() takes 1 positional argument but 2 were giv
en
```

- argümansız olarak veya birden fazla argümanla çağırma işlemi hataya neden olur.

Sum fonksiyonu

- Verilen sayıları toplayan sum() fonksiyonu:

```
sayilar = [45, 90, 43]
print(sum(sayilar))
```

- Liste deki sayılar bu şekilde sum fonk ile toplandı. Hazır olarak listedeki değerleri çarpan fonk yok.
- Nasıl yapılabilir?

Örnek cevabı

```
sayilar = [3, 4, 6]
a = 1
for i in sayilar:
    a = a * i
print (a)
```

- Yazdığımız bu fonksiyonu tek bir kez tanımlayıp ve gereken yerlerde bu fonksiyonu kullanabilir hale gelir.

```python
def carp(liste):
    a = 1
    for i in liste:
        a = a * i
    print(a)

sayilar = [3, 5, 7]
carp(sayilar)
```

İsimli Argümanlar

- Python fonksiyonlarında öğe sıralamasının önem taşıdığı argümanlara “sıralı argüman (positional argument)” adı verilir.
- Argüman sayısının çok fazla olduğu durumlarda isimli argümanları kullanması faydalı olabilir.

```
def islemyap(a, b, c):
    print (a/b)
    print (c)
islemyap(8,4,"merhaba")
#islemyap(8,"merhaba",4)
```

İsimli Argümanlar

```
1   def islemyap(a, b, c):
2       print (a/b)
3       print (c)
4   islemyap(a=8,c="merhaba",b=4)
```

- Böylece argümanları sıra gözetmeden kullanma imkânımız oluyor. Özellikle sayılar ve karakter dizilerini karışık olarak içeren fonksiyonlarda yukarıdaki yöntemin faydası daha belirgindir.

Gömülü Fonksiyonlar (Built-in Functions)

- Hazır fonksiyonlar
  - Range, sum len fonksiyonları örnek olarak verilebilir.
  - Python'daki gömülü fonksiyonların listesi http://docs.python.org/library/functions.html

global Deyimi

```
def fonk():
    a = 5
    print (a)
fonk()
```

- Yukarıdaki örneği çalıştıralım 5 değeri çıktı olarak gelecektir.

```
def fonk():
    a = 5
    print (a)
fonk()
print ("a'nın değeri:",a)
```

global Deyimi

```
Traceback (most recent call last):
  File "main.py", line 5, in <module>
    print ("a'nın değeri:",a)
NameError: name 'a' is not defined
>
```

- “a” diye bir değişken olmadığı hatası

global Deyimi

- Aynı örnek aşağıdaki gibi kullanıldığında;

```
def fonk():
  a = 5
  print (a)
fonk()
a = 10
print ("a'nın değeri: ", a)
```

global Deyimi

```
def fonk():
    global a
    a = 5
    print (a)
fonk()
print ("a'nın değeri: ", a)
```

- Fonksiyon dışından erişim sağlanmış oldu.

global Deyimi

```
a = 10
def fonk():
    a = 5
    return a
print ("a'nın fonksiyon içindeki değeri", fonk())
print ("a'nın fonksiyon dışındaki değeri: ", a)
```

- İlki fonksiyon içindeki “a” değişkeninin değerini gösteriyor.
- İkincisi ise fonksiyon dışındaki “a” değişkeninin değerini
- Python’daki “isim alanı”…

```
a = 10
def fonk():
  global a
  a = 5
  return a
print ("a'nın fonksiyon içindeki değeri", fonk())
print ("a'nın fonksiyon dışındaki değeri: ", a)
```

- “global” deyimi fonksiyon dışındaki “a” değişkenini sildi.

return Deyimi

- Python'da her fonksiyonun bir "dönüş değeri" vardır. Yani Python'daki bütün fonksiyonlar bir değer döndürür.
- Herhangi bir değer döndürmediğimiz fonksiyonlar otomatik olarak "None" diye bir değer döndürür.

```
def fonk():
    a = 5
    print (a)
```

```
def fonk():
    a = 5
    print (a)
print (fonk())
```

pass Deyimi

```
def deneme():
    liste = []
    while True:
        a = input("Giriniz:")
        if a == "0":
            pass
        else:
            liste.append(a)
            print (liste)
deneme()
```

```
Giriniz:8
['8']
Giriniz:7
['8', '7']
Giriniz:6
['8', '7', '6']
Giriniz:4
['8', '7', '6', '4']
Giriniz:0
Giriniz:0
Giriniz:0
Giriniz:
```

```
def bir_fonksiyon():
    pass
```

- Fonksiyon tanımlarken fonksiyon gövdesi boş bırakılmamalı.
- Eğer gövde boş bırakılırsa programı çalışmaz.
- pass deyimi yerleştirilebilir gövde kısmına ne yazacağınıza karar verildiğinde pass deyimi kaldırılabilir.

örnek1

- Bir sayının kendisi dışındaki tüm pozitif bölenlerinin toplamı kendisine eşitse mükemmel sayı olarak adlandırılır. Kullanıcıdan alınan bir sayının mükemmel sayı olup olmadığını kontrol eden bir program yazınız.

örnek1

```
sayi = int(input("Sayi Giriniz = "))
toplam=0
for i in range(1,sayi):
  if(sayi%i == 0):
    print(i)
    toplam = toplam + i
if(sayi==toplam):
  print("Mukemmel Sayidir.")
else:
  print("Mukemmel Sayi Degildir")
```

örnek2

- geçme notu hesaplayan ve sonucu sözlük olarak veren uygulama

```
ögrenci adı giriniz:osman
vize notu giriniz:40
final notu giriniz60
{'ogrenci': 'osman', 'ara_not': 40, 'final_not': 60, 'ortalama'
: 52.0}
```

örnek2

```
def ort_hesapla(isim,ara,final):
    ort = ara*0.4 + final*0.6
    sonuc = { "ogrenci" : isim,
              "ara_not" : ara,
              "final_not" :final,
              "ortalama" : ort}
    return sonuc
isim=input("ögrenci adı giriniz:")
vize=int(input("vize notu giriniz:"))
final=int(input("final notu giriniz:"))
print(ort_hesapla(isim,vize,final))
```

örnek3

- Kullanıcının verdiği bir listedeki elemanların toplamını veren bir fonksiyon yazınız

```
def toplam(liste):
    toplam = 0
    for i in liste:
        toplam = toplam + i
    return toplam
print(toplam([1,2,3,4]))
print(toplam(liste=[1,2,3]))
```

örnek4

- Kendinden küçük tüm asal sayıların listesini ve toplamını veren bir fonksiyon yazınız.

```
True
[1, 11]
[[2, 3, 5, 7], 17]
```

- print(asal_mi(11))
- print(p_bolenler(11))
- print(asallar(11))

örnek4

```
def p_bolenler(n):
    liste = []
    for i in range(1,n+1):
        if (n % i == 0):
            liste.append(i)
    return liste

def asal_mi(n):
    liste = p_bolenler(n)
    if ( len(liste) == 2 ):
        sonuc = True
    else:
        sonuc = False
    return sonuc

def asallar(n):
    liste = []
    for i in range(1,n):
        if (asal_mi(i) == True):
            liste.append(i)
            liste2 = [liste,sum(liste)]
    return liste2
```